	ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		Assignment 1
Branch	**CSE**	Programme	**B.Tech**
Course Name	**Computer Science Workshop 2**	Semester	**4th**
Course Code	**CSE 3141**	Academic Year	**2025-2026**
Assignment: 1		**Topic: Building a Blog Application**	

Learning Level (LL)			
	L1: Remembering	**L3:** Applying	**L5**: Evaluating
	L2: Understanding	**L4**: Analysing	**L6**: Creating

Q's	Questions	COs	LL
Q1.	Develop a Django project named **college_blog** and create an application named **blog** within the project. Configure the application by activating it in the *INSTALLED_APPS* section of *settings.py*. Further, design a basic **Post** model inside *models.py* containing the fields **title** (CharField with max_length=250), **slug** (SlugField with max_length=250), and **body** (TextField). Implement the **__str__()** method to return the title of the post. Perform database migration using the appropriate Django management commands and register the model in the Django administration site. Finally, demonstrate the working of the model by creating at least one Post entry through the admin interface. **Expected Outcomes:** • Access /admin/ • Add a Post object from admin panel	CO1	**L1, L2, L3**
Q2.	Enhance the existing **Post** model of the **blog** application by incorporating additional DateTime fields to support content lifecycle tracking. Add a **publish** field using **DateTimeField** with *default=timezone.now* to record the publishing time of each post. Further, include **created** and **updated** fields using *auto_now_add=True* and *auto_now=True* respectively to automatically track object creation and modification timestamps. Modify the model by defining a **Meta** class that specifies the default ordering of posts in reverse chronological order (newest first) using the **publish** field. Additionally, implement a database index on the **publish** field to improve query performance for sorting and filtering operations. After updating the model, generate and apply the necessary migrations to synchronize the database schema with the updated model structure. **Expected Outcomes:** • Posts are ordered by latest publish date • Database index is created • Admin shows new fields	CO1	**L2, L3, L4**
Q3.	Enhance the existing **Post** model of the **blog** application by incorporating content management and relational database features. Create an inner class named **Status** using Django's **TextChoices** enumeration to define two valid states for a blog post: **DRAFT** and **PUBLISHED**. Add a **status** field to the model using	CO2	**L3, L4**

	CharField, configure it to use the defined choices, and set the default value to **Draft**. Further, establish a many-to-one relationship between the **Post** model and Django's built-in user model by adding an **author** field using **ForeignKey** referencing *settings.AUTH_USER_MODEL*. Configure the relationship with *on_delete=models.CASCADE* to ensure that all posts are deleted if the associated author is removed, and define *related_name='blog_posts'* to enable reverse access from the user model. After modifying the model, generate and apply the required migrations to update the database schema accordingly. **Expected Outcomes:** • Admin shows status dropdown • Post is linked with author • Only valid status values allowed		
Q4.	Enhance the existing **Post** model by implementing advanced data filtering and URL routing mechanisms. Create a custom model manager named **PublishedManager** that overrides the default queryset behavior to return only posts with the status set to **Published**. Integrate this custom manager into the model by defining both the default manager (*objects = models.Manager()*) and the custom manager (*published = PublishedManager()*). Further, implement the **get_absolute_url()** method in the model to generate SEO-friendly URLs following the pattern /year/month/day/slug/, dynamically constructed using the post's publish date and slug. Subsequently, develop the application's view layer by creating a **post_list** view that retrieves and displays only published posts using the custom manager, and a **post_detail** view that fetches a specific post by filtering with year, month, day, and slug parameters. Configure URL routing by creating *blog/urls.py* with appropriate path definitions and include these URLs in the main project's **urls.py** file to establish complete navigation within the application. **Expected Outcomes:** • Visiting / shows only published posts • Clicking a post opens detail page • URL contains date + slug	CO3	**L3, L4**
Q5.	Complete the Blog Application by implementing the presentation layer and advanced administration customization features. Design and organize the template structure under *blog/templates/blog/*, including **base.html**, **post/list.html**, and **post/detail.html**. The **base.html** file should act as the common layout template. The **list.html** template must display the post title, author, publish date, a 30-word preview of the post content, and a hyperlink directing to the respective detail page. The **detail.html** template must present the full content of the selected post along with a navigation link to return to the blog list page. Further, enhance the administrative interface by customizing the **PostAdmin** class to improve usability and data management. Implement the following configurations: *list_display*, *list_filter*, *search_fields*, *prepopulated_fields*, *raw_id_fields*, *date_hierarchy*, *ordering*, and *show_facets*. Ensure that the admin interface provides efficient filtering, searching, sorting, and navigation capabilities. Validate	CO3	**L4, L6**

	that the application functions cohesively, with proper integration between model, views, templates, and URLs, resulting in a fully operational blog system. **Expected Outputs:** • Admin panel fully customized • Blog list page working • Blog detail page working • SEO-friendly URLs working • Only published posts visible		
Q6.	Develop a Django project named **college_project** and create an application named **studentApp** within the project. Configure the application by activating it in the *INSTALLED_APPS* section of *settings.py*. Further, design a basic **Student** model inside *models.py* containing the fields **name** (*CharField* with *max_length=100*) and **age** (*IntegerField*). Implement the **__str__()** method to return the name of the student. Perform database migration using the appropriate Django management commands and register the model in the Django administration site. Finally, demonstrate the working of the model by creating at least one Student entry through the admin interface. **Expected Outcomes:** • Access /admin/ • Add a Student object from admin panel	CO1	**L1, L2, L3**
Q7.	Enhance the existing **Student** model of the **studentApp** application by incorporating additional fields to store more student information. Add a **roll_number** field using *CharField* and a **created** field using *DateTimeField* with *auto_now_add=True* to track when a student record is created. Modify the model by defining a **Meta** class that specifies the default ordering of students in ascending order based on the **name** field. After updating the model, generate and apply the necessary migrations to synchronize the database schema with the updated model structure. **Expected Outcomes:** • Students are ordered alphabetically by name • New fields appear in admin panel • Database table updated successfully	CO1	**L2, L3, L4**
Q8.	Enhance the existing **Student** model by incorporating relational database features. Create a new model named **Department** containing fields **dept_name** (*CharField*) and **dept_code** (*CharField*). Establish a many-to-one relationship between the **Student** model and the **Department** model by adding a **department** field using *ForeignKey* referencing the **Department** model. Configure the relationship with *on_delete=models.CASCADE* and define a suitable *related_name* for reverse access. After modifying the models, generate and apply the required migrations to update the database schema accordingly. **Expected Outcomes:**	CO2	**L3, L4**

	• Student is linked with a Department • Department appears in admin panel • Deleting a Department removes associated students • Relationship is visible in admin interface		
Q9.	Enhance the project by implementing view and template integration mechanisms. Create a template structure under **studentApp/templates/studentApp/**, including **student_list.html**. Develop a **student_list** view that retrieves all student records from the database and renders them using the template. Configure URL routing by adding appropriate path definitions in **college_project/urls.py** to display the student list when accessing **/students/**. Ensure the template dynamically displays the student *name*, *age*, *roll number*, and *department*. **Expected Outcomes:** • Visiting /students/ shows list of students • Student data is dynamically rendered • URL mapping works correctly • Model–View–Template integration verified	CO3	**L3, L4**
Q10.	Complete the Student application by implementing a student detail view and enhancing the administrative interface. Create a **student_detail** view that retrieves a specific student using primary key and displays full details in a new template **student_detail.html**. Update URL routing to support detail view navigation. Further, customize the **StudentAdmin** class by implementing *list_display*, *list_filter*, *search_fields*, and *ordering* to improve usability and data management. Ensure proper navigation between list and detail pages and validate that the application functions cohesively with correct database connectivity and URL routing. **Expected Outputs:** • Student list page working • Student detail page working • Admin panel customized • Navigation between pages functioning correctly • Complete StudentApp application operational	CO3	**L4, L6**
	-END-		

Note:

1. Assignment carries a weightage of 10% **marks out of 100**
2. Only CO1, CO2, and CO3 outcomes are covered.

Course Outcomes	CO1	Remember and identify the fundamental concepts of web development and Django framework.
	CO2	Explain and interpret Django workflows such as authentication, template rendering, and form handling.
	CO3	Apply Django concepts to develop functional web applications.
	CO4	Implement advanced Django features including APIs, payments, internationalization, and real-time systems.
	CO5	Analyze Django applications with respect to caching strategies, API design, data flow, and performance-related design considerations.
	CO6	Analyze and design an end-to-end Django solution including deployment and system integration.